

Jenkins Document

Part A: Theoretical Understanding

Task 1: Continuous Integration (CI) :

1. Introduction to Continuous Integration

Continuous Integration (CI) is a software development practice in which developers frequently integrate their code changes into a shared central repository. Each integration is automatically verified by building the application and running automated tests. The main objective of CI is to detect integration issues early, improve software quality, and reduce the time required to release reliable software.

In traditional development models, developers often worked in isolation for long periods and merged their changes at the end of a development cycle. This approach frequently led to integration conflicts, last-minute bugs, and unstable builds. CI addresses these problems by encouraging small, frequent code commits and continuous validation.

In the modern software development lifecycle, CI has become essential due to fast release cycles, distributed teams, and the growing complexity of applications. Tools like Jenkins automate the CI process and provide fast feedback to developers, ensuring that software remains in a deployable state at all times.

2. Core Principles of Continuous Integration

Continuous Integration is built on several fundamental principles:

2.1 Frequent Code Commits

Developers commit code changes frequently, often multiple times a day. Smaller commits reduce merge conflicts and make it easier to identify the source of bugs.

2.2 Single Source Repository

All source code is stored in a centralized version control system such as Git. This repository serves as the single source of truth for the project.

2.3 Automated Builds

Every code commit triggers an automated build process. This ensures that the application compiles successfully after each change.

2.4 Automated Testing

Automated tests are executed as part of the build process. Unit tests, integration tests, and regression tests help detect defects early.

2.5 Fast Feedback

CI systems provide immediate feedback to developers when a build or test fails. This allows quick fixes and prevents defective code from progressing further.

2.6 Maintain a Deployable State

At any point in time, the codebase should be stable and ready for deployment.

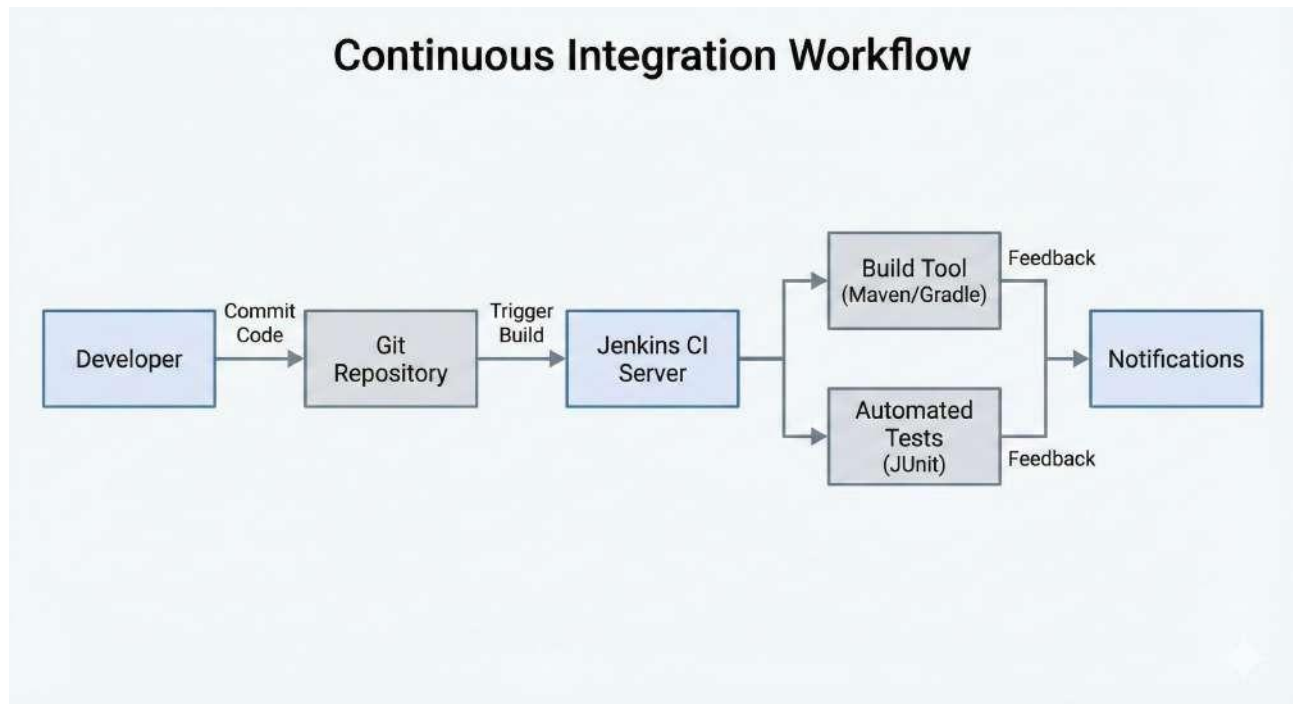


Figure 1: Continuous Integration Workflow

As shown in Figure 1, Continuous Integration automates the build and testing process after every code commit.

3. Difference Between CI, CD, and Continuous Deployment

Although often used together, Continuous Integration, Continuous Delivery, and Continuous Deployment are distinct concepts.

3.1 Continuous Integration (CI)

CI focuses on automatically building and testing code whenever changes are committed to the repository.

Example:

A developer pushes code to GitHub, Jenkins automatically runs unit tests and generates a build report.

3.2 Continuous Delivery (CD)

Continuous Delivery extends CI by ensuring that the software can be deployed to production at any time. Deployment is manual but predictable.

Example:

After successful CI builds, the application is deployed to a staging environment where a release manager approves production deployment.

3.3 Continuous Deployment

Continuous Deployment automates the entire release process. Every successful build is automatically deployed to production without human intervention.

Example:

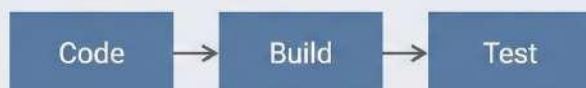
Web applications where updates are released multiple times per day automatically after passing all tests.

3.4 Comparison Table

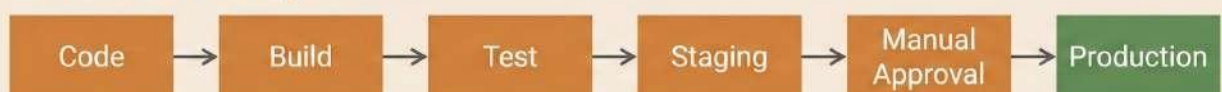
Aspect	Continuous Integration	Continuous Delivery	Continuous Deployment
Focus	Build & Test	Release readiness	Automatic release
Human Approval	Not required	Required	Not required
Risk Level	Low	Medium	High
Automation Level	Partial	High	Very High

CI, Continuous Delivery, and Continuous Deployment Comparison

Continuous Integration



Continuous Delivery



Continuous Deployment

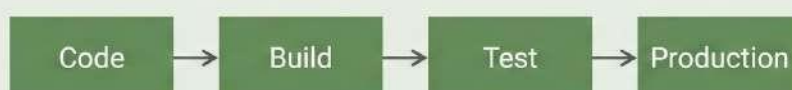


Figure 2: Comparison of Continuous Integration, Continuous Delivery, and Continuous Deployment

Figure 2 illustrates the key differences between Continuous Integration, Continuous Delivery, and Continuous Deployment.

4. Benefits and Challenges of Implementing CI

4.1 Benefits of CI

- Early Bug Detection: Bugs are identified immediately after code changes.
- Improved Code Quality: Automated tests enforce coding standards.
- Faster Development Cycles: Reduced debugging and integration time.
- Better Collaboration: Developers work on a shared codebase.
- Reduced Integration Risk: Frequent merges prevent major conflicts.

4.2 Challenges of CI

- Initial Setup Cost: Requires infrastructure and tool configuration.
- Test Maintenance: Automated tests must be updated regularly.
- Flaky Tests: Unstable tests can cause false failures.
- Learning Curve: Teams need training to adopt CI practices effectively.

5. CI Workflow Components

5.1 Version Control Systems in CI

Version control systems (VCS) such as Git play a central role in CI. They track code changes, manage branches, and enable collaboration. CI tools integrate with VCS to trigger builds whenever code is committed.

5.2 Build Automation Tools

Build automation tools like Maven and Gradle compile source code, manage dependencies, and package applications. They ensure consistent builds across different environments.

5.3 Automated Testing Frameworks

Testing frameworks such as JUnit are used to validate application behavior. Automated tests improve reliability and reduce manual testing efforts.

5.4 Artifact Repository Management

Artifact repositories store compiled binaries and build outputs. They enable versioning, reuse, and controlled distribution of artifacts.

5.5 Notification Systems

Notification systems keep teams informed about build status through emails, dashboards, or messaging tools. Immediate alerts help developers respond quickly to failures.

CI Toolchain and Workflow Components

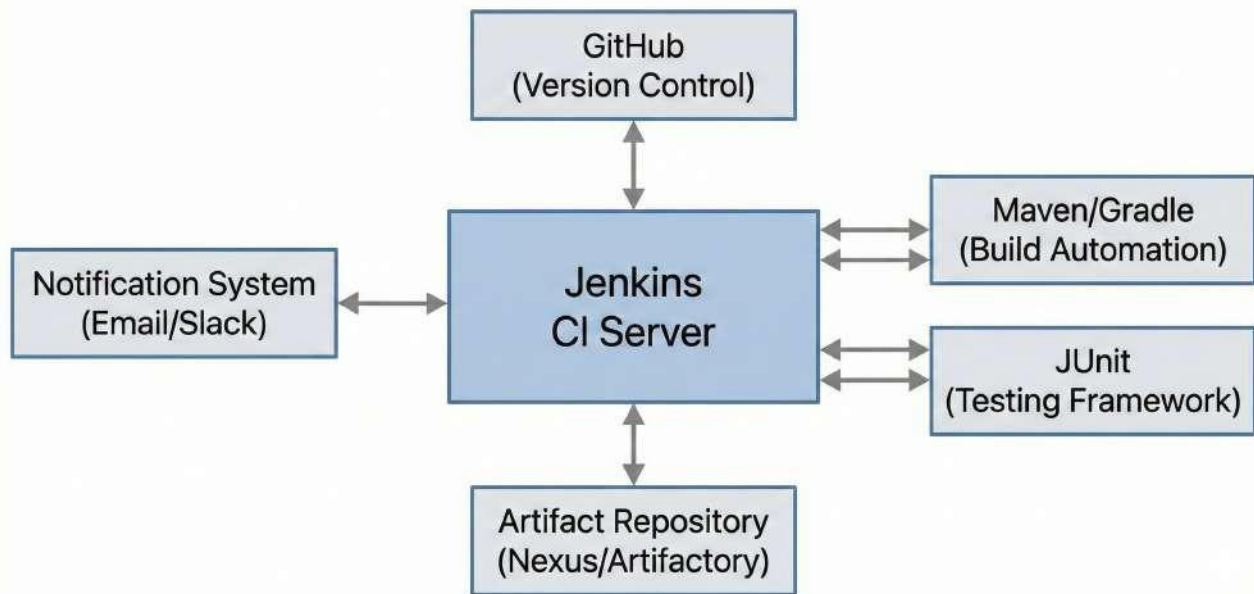


Figure 3: CI Toolchain and Workflow Components The

interaction between CI tools is depicted in Figure 3.

6. Case Study Analysis of CI Implementation

6.1 Organization 1: Netflix

Netflix uses CI to support rapid deployment and scalability. Automated testing and continuous builds allow frequent updates without service disruption. CI helps Netflix reduce downtime and maintain high availability.

6.2 Organization 2: Amazon

Amazon practices CI to support its large-scale e-commerce platform. Every code change is automatically tested, enabling thousands of deployments per day. CI ensures reliability while supporting rapid innovation.

7. Traditional Development vs CI-Based Development

7.1 Traditional Development

- Manual builds and testing
- Late discovery of bugs
- Infrequent releases
- High integration risk

7.2 CI-Based Development

- Automated builds and testing
- Early bug detection
- Frequent, reliable releases
- Reduced integration conflicts

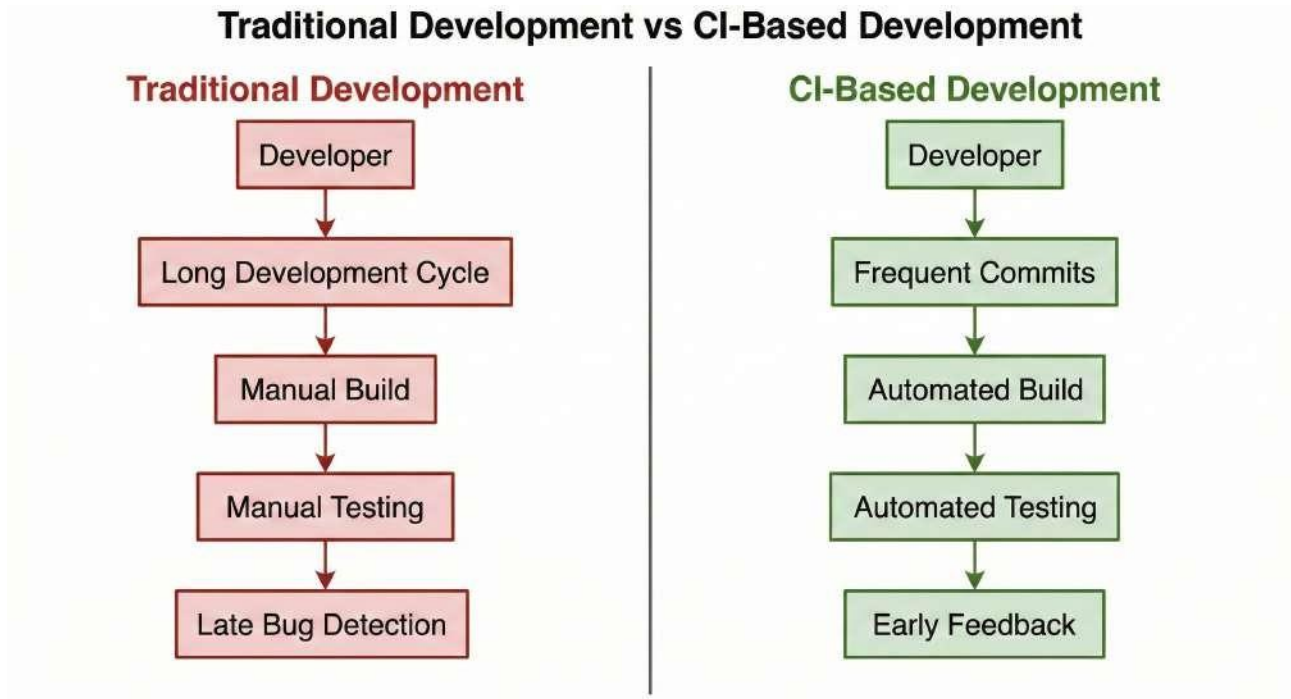


Figure 4: Traditional Development vs CI-Based Development

Figure 4 highlights the contrast between traditional development and CI-based development approaches.

8. Return on Investment (ROI) of CI Implementation

Although CI requires an initial investment in tools and infrastructure, the long-term ROI is significant. CI reduces development time, minimizes production failures, and lowers maintenance costs. Faster releases improve customer satisfaction and give organizations a competitive advantage.

9. Conclusion

Continuous Integration is a foundational practice in modern software development. By automating builds and tests, CI improves software quality, accelerates delivery, and enhances team collaboration. Despite initial challenges, the benefits of CI far outweigh the costs, making it an essential component of DevOps and Agile methodologies.

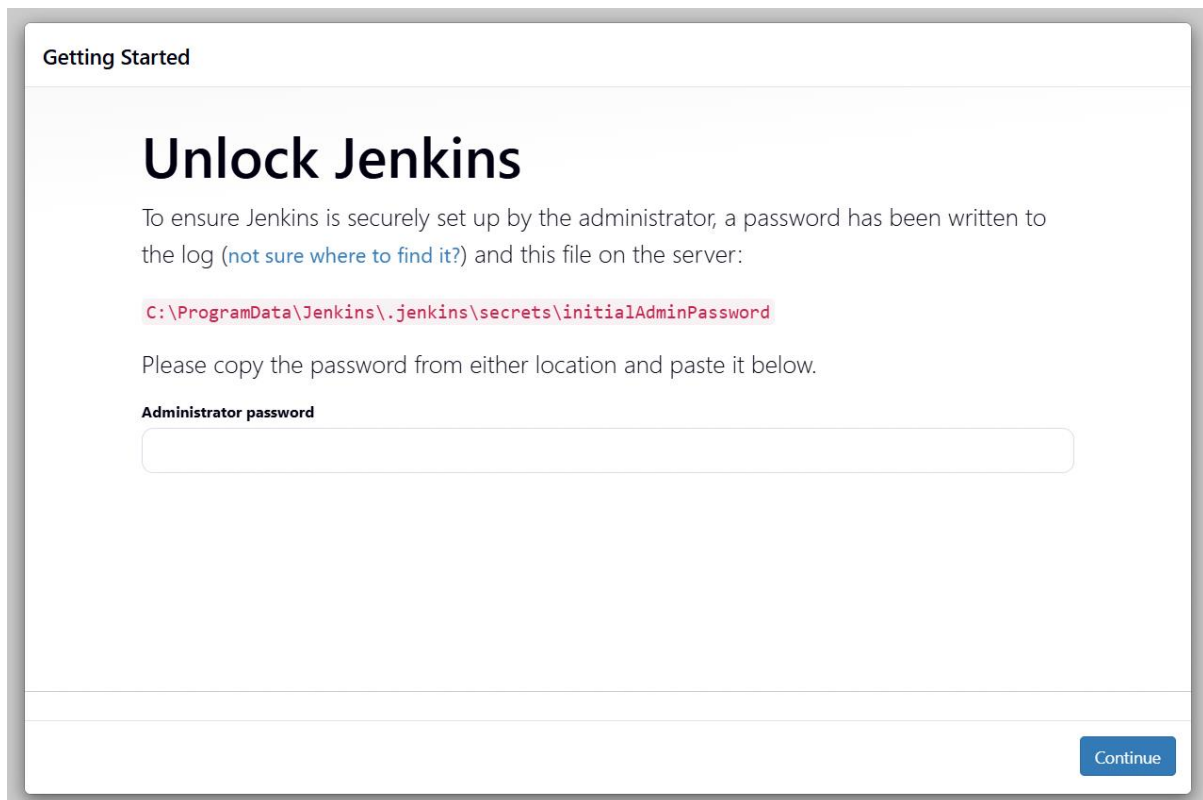
10. References

1. Jenkins Official Documentation
2. Atlassian CI/CD Guide
3. Martin Fowler – Continuous Integration
4. Agile Alliance CI Practices
5. DevOps Handbook

Part B: Jenkins Architecture & Installation

Task 2: Jenkins Installation on Windows

Jenkins was installed on Windows using Eclipse Adoptium.



The image shows the Jenkins 'Getting Started' page for unlocking. It features a title 'Unlock Jenkins', an explanatory paragraph about the administrator password, a code snippet for the password location, a text prompt to copy the password, a text input field labeled 'Administrator password', and a 'Continue' button at the bottom right.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

Figure 2.1: Jenkins Initial Unlock Screen

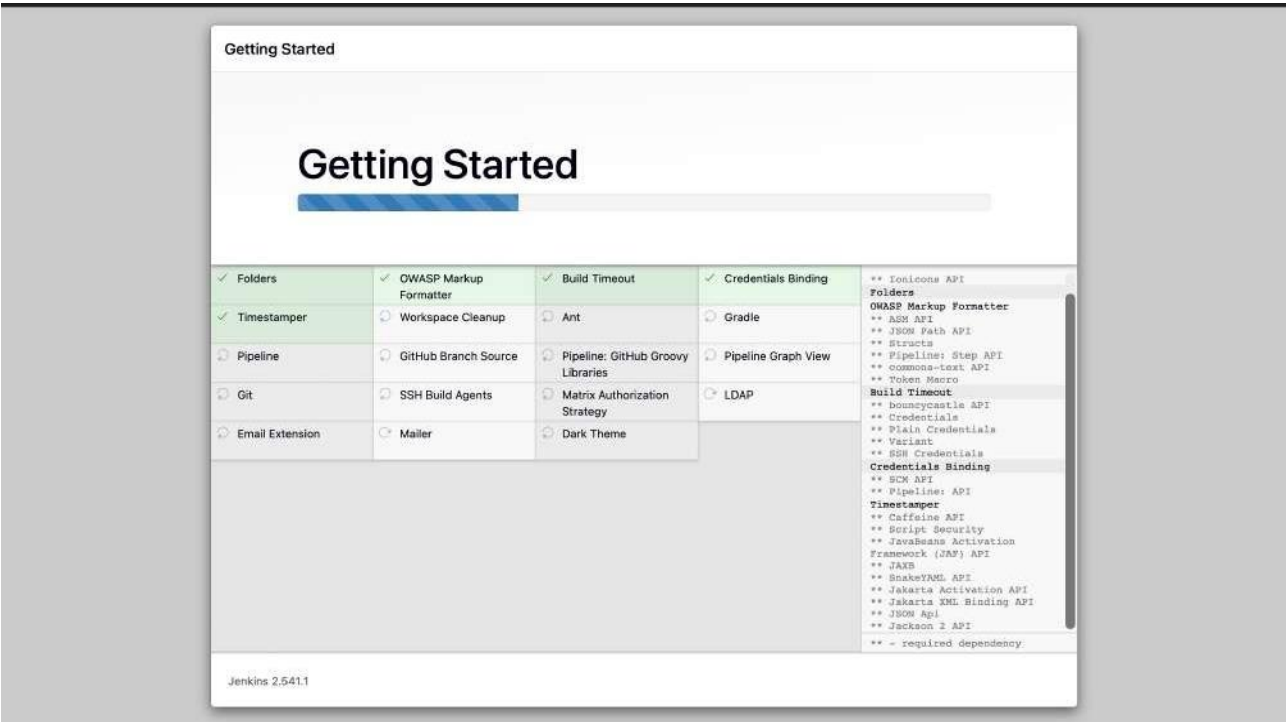
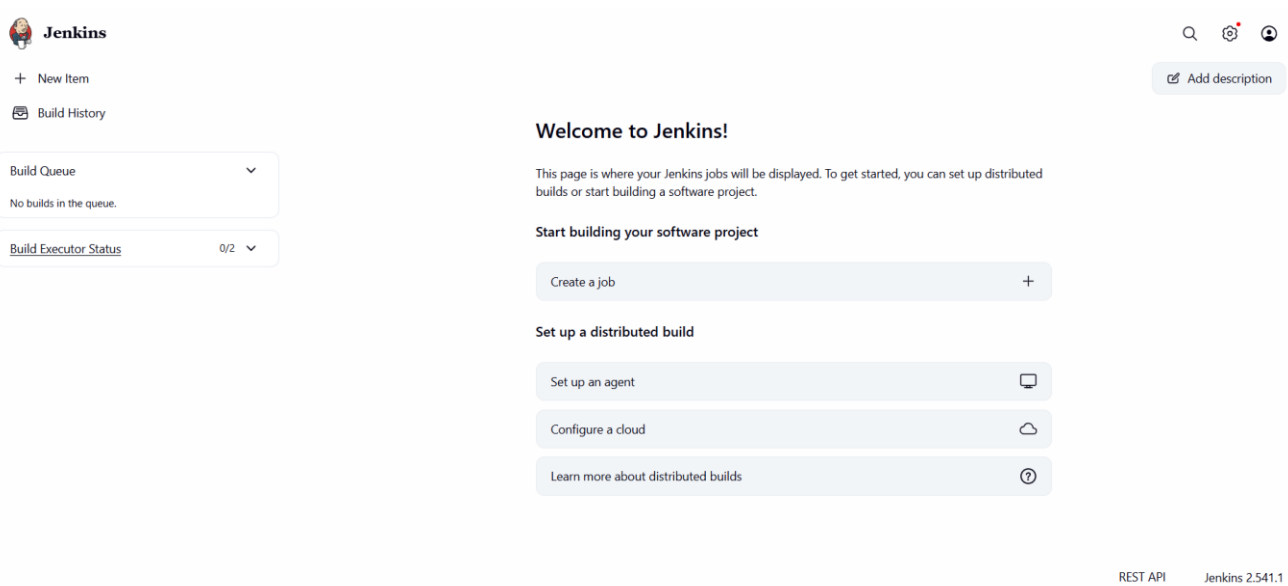


Figure 2.2: Installation of Recommended Jenkins Plugins



Jenkins / Manage Jenkins / Tools

JDK installations

+ Add JDK

⋮ JDK

Name

JDK21

JAVA_HOME

/opt/homebrew/opt/openjdk@21

☐ Install automatically ?

+ Add JDK

Git installations

⋮ Git

Name

Default

Save Apply

Figure 2. Global Tool Configuration (JDK)

Jenkins / Manage Jenkins / Tools

+ Add JDK

Git installations

⋮ Git

Name

Default

Path to Git executable ?

git

☐ Install automatically ?

+ Add Git

Gradle installations

+ Add Gradle

Save Apply

Fig: Global Tool Configuration (git)

Figure: Global Tool Configuration (Maven)

 Jenkins

Manage Jenkins

Tools

Search

Settings

User

Maven installations

+ Add Maven

Maven

Name

Maven3

☒ Install automatically

Install from Apache

Version

3.9.12

+ Add Installer

+ Add Maven

Save

Apply

Jenkins 2.41.1

Figure: Jenkins System Configuration

 Jenkins

Manage Jenkins

System

Search

Settings

User

Jenkins Location

Jenkins URL

http://localhost:8080/

Please set a valid host name, instead of localhost

System Admin e-mail address

address not configured yet <nobody@nowhere>

Serve resource files from another domain

Resource Root URL

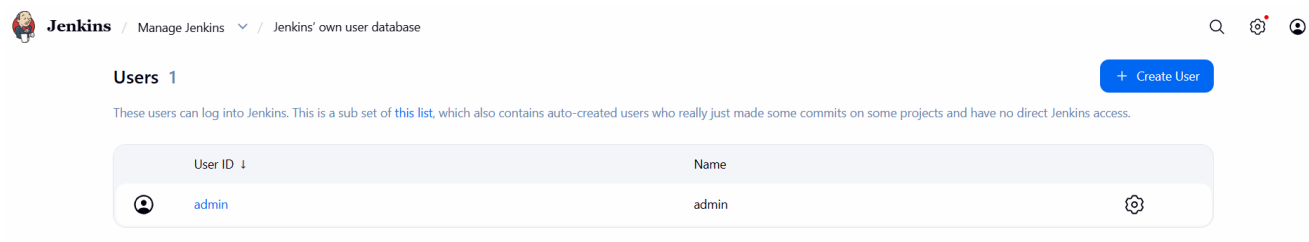
Without a resource root URL, resources will be served from the Jenkins URL with Content-Security-Policy set.

Global properties

Save

Apply

Figure: User Management Screen



Task 3: Jenkins Architecture Diagram

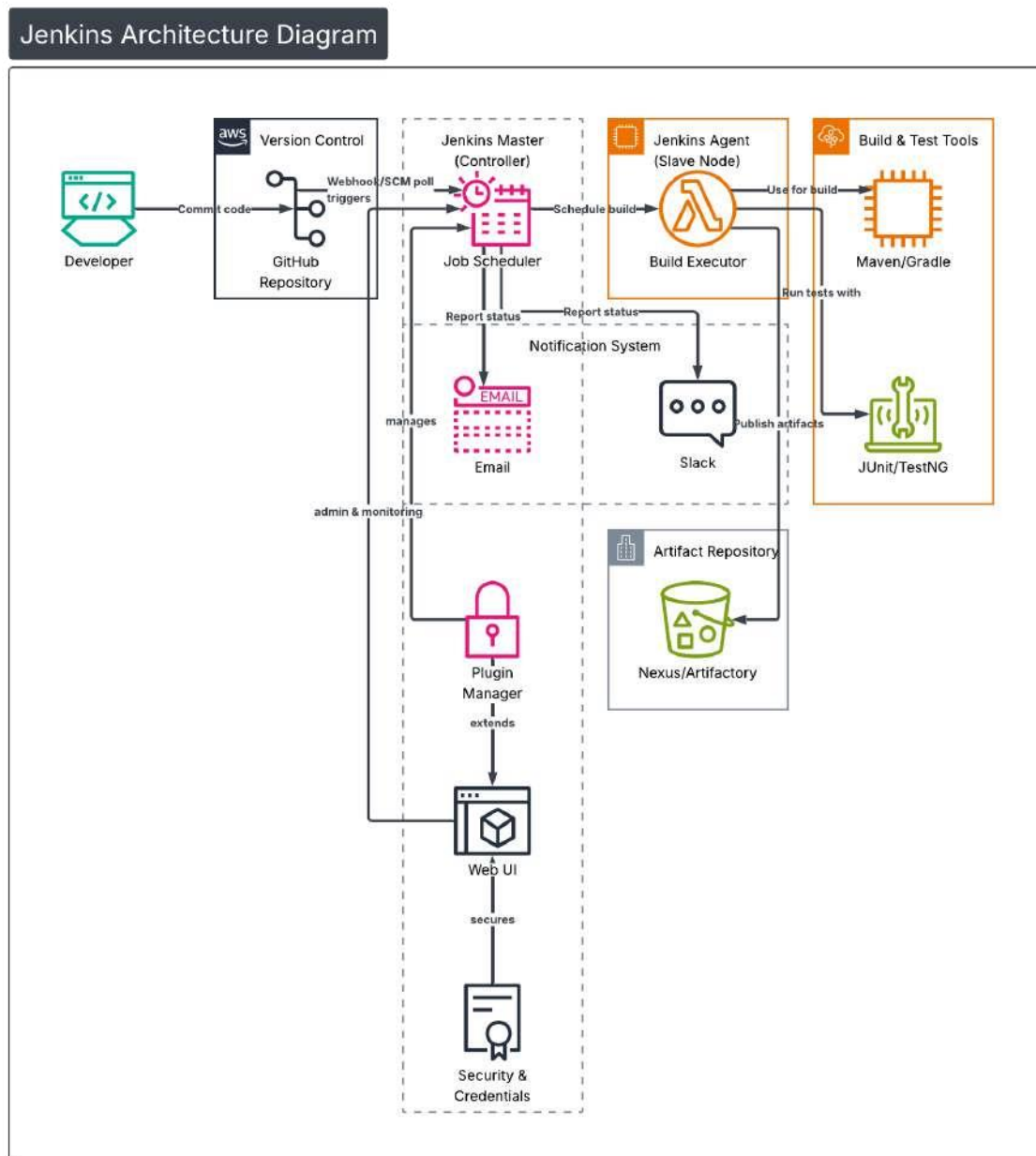


Figure: Jenkins master–agent architecture illustrating CI workflow, tool integrations, plugin ecosystem, artifact management, and notification system.

Jenkins follows a master–agent architecture where the master node manages job scheduling, plugins, security, and user interaction. Build and test tasks are executed on agent nodes to improve scalability. Jenkins integrates with version control systems, build tools, testing frameworks, artifact repositories, and notification systems to provide continuous integration and feedback.

Part C: Jenkins Job Configuration

Task 4: Create Different Types of Jenkins Jobs

Objective

The objective of this task was to configure different types of Jenkins jobs to implement Continuous Integration using both Freestyle and Multibranch Pipeline approaches. The jobs were integrated with a GitHub repository to automatically build, test, and manage artifacts.

Job 1: Freestyle Project

Description

A Freestyle Jenkins job was created to demonstrate a basic CI workflow. The job was configured to pull source code from a GitHub repository, execute build and test steps, and perform post-build actions such as archiving artifacts and publishing test results.

Configuration Details

- Source Code Management: GitHub repository was connected using HTTPS.
- Build Triggers:
Manual build trigger was used (suitable for demonstration).
- Build Steps:
 - Maven build command executed:
mvn clean test
 - This step compiled the project and executed JUnit test cases.
- Post-build Actions:
 - Archive Artifacts: Build artifacts were archived for future reference.
 - Publish JUnit Test Results: Test reports were published using the path:
**/target/surefire-reports/*.xml

Result

The Freestyle job executed successfully. The build completed without errors, and all JUnit tests passed. Archived artifacts and test reports were visible in the Jenkins dashboard.



Jenkins

Freestyle-CI-Job

Configure



Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Trigger builds remotely (e.g., from scripts) ?

☐ Build after other projects are built ?

☐ Build periodically ?

☒ GitHub hook trigger for GITScm polling ?

☒ Poll SCM ?

Schedule ?

H/5 * * * *

Would last have run at Sunday, February 1, 2026, 7:08:00 PM India Standard Time; would next run at Sunday, February 1, 2026, 7:13:00 PM India Standard Time.

☐ Ignore post-commit hooks ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

☐ Delete workspace before build starts

...

Save

Apply



Jenkins

Freestyle-CI-Job

Configure



Configure

General

Source Code Management

Triggers

Environment

Build Steps

Post-build Actions

☐ Respect branching for pre-build tasks ?

☐ Terminate a build if it's stuck

☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets ?

Maven Version

(Default)

Goals

clean test

Advanced

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

+ Add post-build action

Save

Apply



Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Previous Build

#2 (Feb 1, 2026, 6:37:19 PM)



[Build Artifacts](#)

[build-output.txt](#) 20 B [view](#)



Started by user [admin](#)



No changes.



Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Publish JUnit test result report

Test report XMLs

Fileset 'includes' setting that specifies the generated raw XML report files, such as 'myproject/target/test-reports/*.xml'. Basedir of the fileset is the workspace root.

Test output retention

None

☐ keep all the properties

☐ Keep original test names, even when reporting from multiple stages

Health report amplification factor

1.0

1% failing tests scores as 99% health. 5% failing tests scores as 95% health

Allow empty results

☐ Do not fail the build on empty test results

Skin publishing checks

Save

Apply



Jenkins

Freestyle-CI-Job / #2



Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Previous Build

Console

Download

Copy

View as plain text

```
Started by user admin
Running as SYSTEM
Building in workspace C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job
[Freestyle-CI-Job] $ cmd /c call C:\WINDOWS\TEMP\jenkins17000833482114052388.bat

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>echo =====
=====

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>echo Jenkins Freestyle Job Run
Jenkins Freestyle Job Run

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>echo =====
=====

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>echo Build executed successfully
Build executed successfully

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>echo Creating artifact 1>build-output.txt

C:\ProgramData\Jenkins\.jenkins\workspace\Freestyle-CI-Job>exit 0
Archiving artifacts
Finished: SUCCESS
```


Job 2: Multibranch Pipeline

Description

A Multibranch Pipeline job was configured to automatically discover branches from the GitHub repository and execute a pipeline defined in a Jenkinsfile. This job demonstrates an advanced CI setup that supports multiple branches with automated detection and execution.

Configuration Details

- Branch Source:
GitHub repository connected using a Personal Access Token (PAT) to avoid API rate limits.
- Branch Discovery:
Jenkins automatically detected the main branch.
- Pipeline Definition:
The pipeline was defined using a declarative Jenkinsfile stored in the repository.
- Pipeline Stages:
 - Checkout: Source code was checked out from GitHub.
 - Build & Test: Maven was used to build the project and run JUnit tests.
- Tools Used:
 - Maven (configured as Maven3)
 - JDK 21

Visualization of Stages

Pipeline stages were visualized using the Pipeline Overview view in Jenkins, which displays stage- wise execution for declarative pipelines in newer Jenkins versions.

Result

The Multibranch Pipeline successfully indexed the repository, detected the main branch, and executed the pipeline. The build completed successfully with no test failures, confirming proper CI execution across branches.

Conclusion

Through Task 4, both Freestyle and Multibranch Pipeline jobs were successfully implemented in Jenkins. The Freestyle job demonstrated a simple CI workflow, while the Multibranch Pipeline showcased automated branch discovery and staged pipeline execution. Together, these jobs illustrate the practical use of Jenkins for Continuous Integration in real-world software development.

Jenkins

Multibranch-CI-Pipeline

Configuration

Configuration

General

Branch Sources

Build Configuration

Scan Multibranch Pipeline Triggers

Orphaned Item Strategy

Appearance

Health metrics

Properties

Behaviors

Discover branches ?

Strategy ?

All branches

Discover pull requests from origin ?

Strategy ?

The current pull request revision

Discover pull requests from forks ?

Strategy ?

The current pull request revision

Trust ?

From users with Admin or Write permission

May not be supported on older versions of GitHub Enterprise. See help button.

Add

Property strategy

Save

Apply

Jenkins

Multibranch-CI-Pipeline

main

Status

Changes

Build Now

View Configuration

Stages

GitHub

Pipeline Syntax

main

Full project name: Multibranch-CI-Pipeline/main

Permalinks

Last build (#1), 2 min 30 sec ago

Last failed build (#1), 2 min 30 sec ago

Last unsuccessful build (#1), 2 min 30 sec ago

Last completed build (#1), 2 min 30 sec ago

Builds

Filter

Today

#1

7:46 PM

REST API

Jenkins 2.541.1

Jenkins

Multibranch-CI-Pipeline

main

#2

Tests

Status

Changes

Console Output

Edit Build Information

Delete build '#2'

Timings

Git Build Data

Tests

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Tests 1

✓ 1 Took 21 ms

All tests are passing!
Nice one! All 1 tests are passing.

All Tests

Package	Failed	Skipped	Passed	Total	Duration
com.mu.j.ci	0	0	1 +1	1 +1	1 ms

REST API Jenkins 2.541.1

Jenkins

Multibranch-CI-Pipeline

main

#2

Stages

< #2

Rerun

main 167d2d1 aryan00719/CiLabProject Started by branch indexing Started by Aryan Mishra Started 6 min 53 sec ago Queued 8.4 sec Took 10 sec Changes

Tests

Graph

Start Checkout SCM Tool Install Checkout Build & Test Post Actions End

Q Search

✓ Checkout SCM 2s

✓ Tool Install

✓ Checkout 2s

✓ Build & Test 2s

✓ Post Actions 82ms

✓ Post Actions 82ms Started 7m ago Jenkins

✓ Archive JUnit-formatted test results **/target/surefire-reports/*.xml 44ms

0 Recording test results

1 [Checks API] No suitable checks publisher found.

Jenkins 2.541.1

Step	Arguments	Status
Start of Pipeline - (7.7 sec in block)		Success
node - (7.6 sec in block)		Success
node block - (7.5 sec in block)		Success
stage - (2.1 sec in block)	Declarative: Checkout SCM	Success
stage block (Declarative: Checkout SCM) - (2 sec in block)		Success
checkout - (1.9 sec in self)		Success
withEnv - (5.3 sec in block)	GIT_BRANCH, GIT_COMMIT, GIT_PREVIOUS_COMMIT, GIT_URL	Success
withEnv block - (5.3 sec in block)		Success
stage - (0.32 sec in block)	Declarative: Tool Install	Success
stage block (Declarative: Tool Install) - (0.21 sec in block)		Success
tool - (33 ms in self)	Maven3	Success
envVarsForTool - (92 ms in self)		Success
tool - (34 ms in self)	JDK21	Success
envVarsForTool - (28 ms in self)		Success

Task 5: Plugin Management

Objective

The objective of this task was to enhance Jenkins functionality by installing and managing essential plugins required for source control integration, build automation, testing, notifications, and deployment.

Installed Plugin Categories

Pipeline Plugins

- Pipeline
- Blue Ocean

These plugins enable declarative and scripted pipelines and provide a modern UI for visualizing pipeline execution.

Source Code Management (SCM) Plugins

- Git
- GitHub
- Subversion

These plugins allow Jenkins to integrate with version control systems and automatically trigger builds based on repository changes.

Build Tool Plugins

- Maven Integration
- Gradle
- Ant

These plugins support automated builds using industry-standard build tools.

Testing Plugins

- JUnit
- JaCoCo
- Cobertura

These plugins enable test execution, test report publishing, and code coverage analysis.

Notification Plugins

- Email Extension
- Slack Notification

These plugins allow Jenkins to notify users about build results through email and messaging platforms.

Deployment Plugins

- Docker
- Kubernetes
- SSH

These plugins support application deployment and remote execution in containerized and server environments.

Verification

All required plugins were successfully installed and verified through the Jenkins Plugin Manager. Successful builds confirmed that the plugins were correctly configured and functioning as expected.

Artifacts and Evidence

- Screenshots of installed plugins with versions
- Console output of successful Freestyle and Multibranch pipeline builds
- Jenkins dashboard and job configuration screenshots

Conclusion

Task 5 demonstrates effective plugin management in Jenkins. By installing and organizing essential plugins, Jenkins was extended to support a complete CI workflow, including SCM integration, automated builds, testing, notifications, and deployment readiness.

 Jenkins

Manage Jenkins

Plugins

Search

Settings

Menu

User

Plugins

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Download progress

Preparation

- Checking internet connectivity
- Checking update center connectivity
- Success

commons-lang3 v3.x Jenkins API

Success

Icons API

Success

Folders

Success

OWASP Markup Formatter

Success

ASM API

Success

JSON Path API

Success

Structs

Success

Pipeline: Step API

Success

commons-text API

Success

Token Macro

Success

Build Timeout

Success

bouncycastle API

Success

Credentials

Success

Plain Credentials

Success

Variant

Success

SSH Credentials

Success

Credentials Binding

Success

SCM API

Success

Pipeline: API

Success

Timestamp

Success

Caffeine API

Success

Script Security

Success

 Jenkins

Manage Jenkins

Plugins

Search

Settings

Menu

User

Plugins

Updates

Available plugins

Installed plugins

Advanced settings

Download progress

Search installed plugins

Name	Health	Enabled	
Ant Plugin 520.vd082ecfb_16a_9 Adds Apache Ant support to Jenkins Report an issue with this plugin	96		
Apache HttpComponents Client 4.x API Plugin 4.5.14-269.vfa_2321039a_83 Bundles Apache HttpComponents Client 4.x and allows it to be used by Jenkins plugins. Report an issue with this plugin	96		
Apache HttpComponents Client 5.x API Plugin 5.6-183.v65a_8a_b_e71e59 Bundles Apache HttpComponents Client 5.6 and allows it to be used by Jenkins plugins. Report an issue with this plugin	100		
ASM API Plugin 9.0.1-189.vb_5ef2964da_91 This plugin provides the ASM APIs (v9.1) for other plugins. Report an issue with this plugin	100		
Authentication Tokens API Plugin 1.144.v5f14a_5ec5c33 This plugin provides an API for converting credentials into authentication tokens in Jenkins. Report an issue with this plugin	100		
Bitbucket Branch Source Plugin 937.2.3 Allows to use Bitbucket Cloud and Bitbucket Server as sources for multi-branch projects. It also provides the required connectors for Bitbucket Cloud Team and Bitbucket Server Project folder (also known as repositories auto-discovering). Report an issue with this plugin	96		
Bitbucket Pipeline for Blue Ocean 1.27.25 BlueOcean Bitbucket pipeline creator Report an issue with this plugin	97		

